



Asynchrony and GPUs: Bridging this Dichotomy for I/O with AGIO

Jihoon Han
The Pennsylvania State University
University Park, PA, USA
jhan@psu.edu

Anand Sivasubramaniam
The Pennsylvania State University
University Park, PA, USA
axs53@psu.edu

Chia-Hao Chang
Nvidia
Santa Clara, CA, USA
chiahaoc@nvidia.com

Vikram Sharma Mailthody
Nvidia Research
Santa Clara, CA, USA
vmailthody@nvidia.com

Zaid Qureshi
Nvidia Research
Santa Clara, CA, USA
zqureshi@nvidia.com

Wen-Mei Hwu
Nvidia Research
Santa Clara, CA, USA
whwu@nvidia.com

Abstract

GPUs rely on a largely synchronous programming and execution model. With increasing need to access data residing on SSDs, GPU threads can incur significant latencies for such accesses when using blocking/synchronous I/O mechanisms. There is little hardware/systems support today to perform non-blocking/asynchronous operations from GPU threads directly to tolerate microsecond level latencies incurred in SSD accesses. To fix this dichotomy, this paper presents the design, implementation and evaluation of AGIO, which provides APIs and a runtime environment for GPU threads to directly perform asynchronous I/O operations (fully GPU-orchestrated without CPU involvement). AGIO decouples, both in time and space, the I/O initiation from its completion, to allow useful computation in-between, in order to hide much of the I/O latency. This is particularly useful in applications with access patterns known at compile time, where similar to prefetching, AGIO I/Os can be introduced ahead of need, to yield 65% better performance than its synchronous counterpart. A non-intuitive benefit of AGIO, particularly in applications with data dependent accesses, is the ability to allow threads to proceed beyond I/O initiation, towards initiating more I/O, even if there is little compute to overlap. Such pro-active I/O issuance increases the I/O parallelism to more fully utilize underlying bandwidths, yielding 32% better performance than the synchronous alternative in data dependent executions. Decoupling initiation from completion makes AGIO more adaptive to dataset characteristics, with the programmer not needing a priori knowledge of inputs for effective performance. We also show that AGIO can meet (or better) the performance using a GPU with fewer than half the compute engines of its synchronous counterparts.



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '26, Pittsburgh, PA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2359-9/2026/03

<https://doi.org/10.1145/3779212.3790130>

CCS Concepts: • Computer systems organization → Architectures; • Information systems → Storage architectures; • Computing methodologies → Concurrent computing methodologies.

Keywords: GPUs; Storage; SSDs; Asynchrony; GPUDirect; I/O; P2P

ACM Reference Format:

Jihoon Han, Anand Sivasubramaniam, Chia-Hao Chang, Vikram Sharma Mailthody, Zaid Qureshi, and Wen-Mei Hwu. 2026. Asynchrony and GPUs: Bridging this Dichotomy for I/O with AGIO. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26)*, March 22–26, 2026, Pittsburgh, PA, USA. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3779212.3790130>

1 Introduction

GPUs inherently rely on a steady stream of input data to keep their throughput-oriented plentiful compute engines busy. While on-chip caches and high-bandwidth memories address this need by exploiting data locality, current and future big data applications have working sets that can spill well beyond these capacities, into host memory and even storage devices. Further, caches and GPU memories cannot reduce compulsory misses to cold data residing on storage devices. There have been recent efforts to optimize data movement between GPU memory and other server repositories, whether host memory or SSDs. However, high latencies for such transfers can get into the critical path of GPU computations. This is particularly acute for storage (SSD) accesses that run into microseconds, with little hardware support on today's GPUs to hide such high latencies. Recognizing these limitations, this paper presents the design, implementation, and evaluation of AGIO, an API and runtime system to support asynchronous I/O directly from GPU threads.

GPUs use a largely synchronous programming and execution model by design, where thread level parallelism and context switching across threads are the key knob to hide data access latencies. Until recently, GPUs were restricted to accessing their own memory (on the GPU card), with

multithreading and simple asynchronous memory transfer support (e.g. TMA [34]), largely sufficing to hide such latencies spanning dozens/hundreds of cycles. However, such mechanisms are woefully inadequate to address microsecond level latencies incurred when accessing data residing on storage devices (SSDs). As many related studies note, it is imperative for GPUs to access data beyond their local memories, reaching into host memories and even SSDs, because of the large working sets in many big data applications.

Consequently, there are proposals to extend the reach of GPU threads to SSDs [32, 40]. However, these rely on handing off the requests to the host CPU, where drivers perform the operation to/from I/O devices. Many early mechanisms [40] in this category involved multiple stages of data movement - from SSDs to host memory and then subsequently to GPU memory, or vice-versa. To reduce such data movements, subsequent mechanisms (e.g. [32]) resorted to “data-direct” transfers, where data moves directly between SSDs and GPU memory without employing host memory as an intermediate. However, the control/orchestration of these transfers is still initiated by the host CPU, i.e. GPU threads hand off the operation to host drivers.

Method	Orch.	Move Between	Exp?	Sync?	Time
TMA[34]	GPU	GPUMem-Cache	Exp.	Async	cycle
UVM[31]	CPU	GPUMem-CPUMem	Imp.	Sync	cycle
GPUfs[40]	CPU	GPUMem-CPUMem-SSD	Exp.	Sync	μ s
GDS[32]	CPU	GPUMem-SSD	Exp.	Async.	μ s
BaM[37]	GPU	GPUMem-SSD	Imp.	Sync	μ s
AGIO	GPU	GPUMem-SSD	Exp.	Async.	μ s

Table 1. Data Movement (Exp./Imp. = Explicit/Implicit)

Much more recent work [37] notes the inefficiencies of such CPU orchestration/control, even if data is “directly” transferred. Instead, they directly employ GPU threads to program NVMe queues to perform both control- and data-direct, with no CPU involvement at all.

However, in a “synchronous” programming and execution model as in [37], GPU threads initiate the transfer, but then wait for data to arrive in its memory before resuming execution. Even on a normal CPU, the drawbacks of blocking I/O are well known - unless there is overlap by switching to other threads/processes, there will be gross underutilization. This problem becomes more acute for GPUs because: (i) a GPU compute engine (of a Streaming Multiprocessor, SM) typically holds a few dozen contexts (64 warps on the Nvidia A100 SM) of a single application for multithreading, making it difficult to hide microsecond level I/O latencies; (ii) with the SIMT (Single Instruction Multiple Thread) programming model, these multiple threads are not usually independent/orthogonal (they are likely to hit I/O points in a synchronized fashion), making it difficult to find threads that can proceed with computation while some others are waiting for

I/O, unlike independent processes on a multiprogrammed CPU; (iii) the SIMT model can also create work imbalance within a warp - if not all threads have an equal amount of I/O, or some I/Os hit in the cache while others do not - making all the threads of a warp run slowly.

It is thus imperative to support an *asynchronous programming model* and *runtime system* for I/O on GPUs, which are otherwise based on a synchronous model/system (hence, the *dichotomy*). This is the goal of AGIO, where we provide explicit asynchronous I/O APIs for GPU threads, and implement an efficient runtime system to allow them to make direct and explicit asynchronous I/O calls with the potential of overlapping I/O latencies with useful computation. We implement AGIO on a Nvidia A100, without requiring additional hardware support, and show its benefits over synchronous I/O (and other CPU-orchestrated asynchronous mechanisms) using a diverse spectrum of static (data access patterns known a priori) and dynamic (runtime data dependencies impact the I/O) applications. Along the way, this paper makes the following contributions:

- Decoupling I/O initiation from the subsequent wait/consumption, allows threads to overlap some of their computations with ongoing I/O, for better GPU utilization. This is particularly useful in static applications (e.g. dense matrix computations), where programmers/compiler can explicitly initiate I/O a priori (similar to prefetching) - controlling both the granularity and the time - while almost completely hiding I/O latency with computations on data already obtained. Across our static applications, AGIO provides 1.65x average speedup over state-of-the-art GPU-orchestrated synchronous I/O mechanisms [37].
- Even if there is no computation overlap, decoupling initiation from the wait, allows threads initiating I/O to proceed and possibly initiate more I/O. This can result in better I/O parallelism (p) to more fully utilize the underlying hardware I/O bandwidths, that would otherwise be underutilized. With synchronous I/O, threads blocking for I/O cannot issue more I/O until the prior operations are done. Further, with the SIMT nature of GPUs, even if a thread does not perform I/O, it may be slowed down by another thread of the same warp needing I/O. These effects are more pronounced in dynamic (pointer-chasing) applications where the I/O parallelism can vary quite significantly during execution, stressing the importance of asynchronous I/O in such cases. As a result, AGIO provides 1.32x speedup on average over synchronous I/O mechanisms in these dynamic applications.
- Apart from decoupling (in time) the initiation from wait/consumption, AGIO also allows any thread (decoupling in space) to do the latter, i.e. the initiation and wait/consumption need not be done by the same thread. Such flexibility enables better load balance across threads of a warp, as well as the load across warps. This leads to better GPU

utilization overall. In fact, this decoupling in both time and space allows AGIO to provide as good performance, or even better in many cases, with a GPU that is less than half the size (48 SMs) compared to the synchronous alternatives running on a full Nvidia A100 with 108 SMs.

- A key benefit of AGIO is its adaptability to dataset characteristics (in dynamic applications) without a priori knowledge and tuning that other mechanisms require. Current mechanisms depend on effective (a priori) work assignment to threads - allotting edges of the same graph vertex to threads of a warp or assigning them to entirely different warps, with both having their pros and cons. AGIO, on the other hand, performs well regardless of such work assignments, dynamically adapting to dataset characteristics. This is an important advantage for programmers in the real world, where dataset characteristics may not be readily available for a priori optimizations.
- There are vendor offerings (e.g. Nvidia's CUDA streams [30]) to run GPU kernels asynchronously, which can be indirectly used to implement asynchronous I/O with otherwise blocking I/O APIs such as GDS (GPUDirect Storage, cuFile [32]). However, such mechanisms (i) still use the CPU for orchestrating the I/O, which can be inefficient; (ii) have additional kernel launch/management overheads; and (iii) are mainly useful for applications with data accesses known before kernel launch time. Even for the applications that can use these indirect mechanisms, we show that AGIO is a much more efficient alternative, coming close to the I/O bandwidth that the hardware can sustain.

GPU-initiated asynchronous transfers with AGIO are important not just for dynamic, data-dependent I/O requests (allowing GPUs to directly initiate transfers during execution), but also for a priori known (or even one-time) accesses whose data movement can be time-consuming enough to benefit from overlap with useful GPU computation.

2 Programming Model for AGIO

We want our Asynchronous I/O API to be relatively simple for programming, not very different from those used today in non-GPU contexts (e.g. Linux asynchronous I/O [21]) or even on GPUs (e.g. asynchronous memory copies as in `cuda::memcpy_async()` [33]). At the same time, there are nuances about GPU applications and the underlying (highly multithreaded SMT) execution model that may necessitate some modifications for efficiency. Keeping this dichotomy in mind, we discuss below our APIs for initiation and completion of asynchronous I/O on GPUs.

Initiation: In Linux asynchronous I/O, a thread constructs a control block with details such as the byte range, offset, and target location of an I/O operation. It then issues the request asynchronously using `aio_read(cb)`. This control block (cb) serves as a handle, allowing the initiating thread to query the status or resubmit the operation. Our APIs for

```
1 struct ctrl_blk {
2     void *buf, int64 offset, n_bytes; int32 devid;
3     app_data *app_ptr, int64 cid, int32 tag, ...
4 }
```

Listing 1. I/O control block allocated on thread-local stack.

initiation of asynchronous I/O on GPUs are quite similar, defined by the functions - `g_ao_read()`, `g_ao_write()` - shown in Table 2. A thread builds the cb containing the usual parameters for performing the I/O (Listing 1). The two asynchronous functions for initiating read and write I/O operations are provided with this control block. `cid` and `tag` will be explained below.

Completion/Consumption: Once initiated, there are different options to be notified of I/O completion: (i) via an interrupt/signal to invoke a handler function; and/or (ii) explicitly waiting for the operation to be done. In our current system, we only support the latter (using `g_wait()`) which suffices for many GPU applications, and is also used by some asynchronous GPU memory copy functionalities (similar to `wait_barrier()` in TMA transfers).

Still, applications may want different wait semantics based on who is waiting and what they are waiting for (Figure 1). Although our API does not impose strict thread affinity, it is quite common (in normal applications) for the same thread that initiates the I/O, to also wait for its completion. Below (Case 1), we discuss such an interface where initiation/completion is done by the same thread. However, in high-throughput environments with thousands of threads and hundreds of concurrent I/O operations, such a paradigm can become a bottleneck. In Case 2, we present an option to decouple the issuing and waiting threads for better utilization. Finally, such decoupling can create problems in certain applications where we want a group/set of I/O completions to be handled by the same thread, and in Case 3, we show how our API can be leveraged for such purposes.

2.1 Case 1: Decoupled I/O (in Time) - Same thread for Initiation & Consumption

Many workloads, as in dense matrix computations, are relatively deterministic in what data will be accessed and when. The load can be balanced a priori, and the same thread(s) can be employed for I/O initiation and consequent data consumption for maximum overlap of computation with I/O latencies. In fact, resorting to dynamic determination of initiator/consumer can lead to higher overheads. In such cases, a thread can initiate I/O using `g_ao_read()` for say a row of the matrix for the next iteration (i.e. a prefetch), wait for the completion of the read from the previous iteration (using `g_wait((cid,*))` - the specific read that this same thread initiated), and perform the computation on that data. In this example, the same thread is the initiator as well as the consumer for the I/O (Figure 1 (a)). This can work reasonably

well if the work can be well balanced across all the threads, and the overlap of computation can hide the I/O latencies (so `g_wait()` returns almost immediately).

2.2 Case 2: Decoupled I/O (in Time and Space) - Different threads for Initiation & Consumption

Consider a dynamic access pattern application like BFS, where threads traverse graph nodes residing on SSDs. There is dynamism in what data is read (based on pointers from a vertex), and how much data is read (number of child vertices) at each step. If a thread initiates an I/O for its children and waits, there is a possibility of waiting even if some other pending I/O is done (other vertices), leading to underutilization. In such applications, it would be better to decouple the initiation thread from the completion/consumption thread (Figure 1 (b) - any other thread can call `g_wait((*,*))`, rather than the initiator). However, such decoupling introduces additional considerations in the application implementation and APIs.

The initiator and consumer may need to exchange data, e.g. the consumer may need to know who the parent for this I/O is, and how many children, from the initiator. Since the consumer and initiator are different, these need to be explicitly communicated to the former by the latter. In normal (CPU) programs, it is natural to use shared address spaces and locks between threads. In GPUs, even if there is GPU global memory, its accesses are much more expensive. We need to dynamically exchange information between initiator and consumer (since it is not known a priori), which is provided through our dynamic memory allocator (`g_get/free_mem()`) (see Section 3.3). The use of such a mechanism is illustrated in Listing 2 of Appendix A.1, along with the mechanism for decoupling the initiation and consumption across threads.

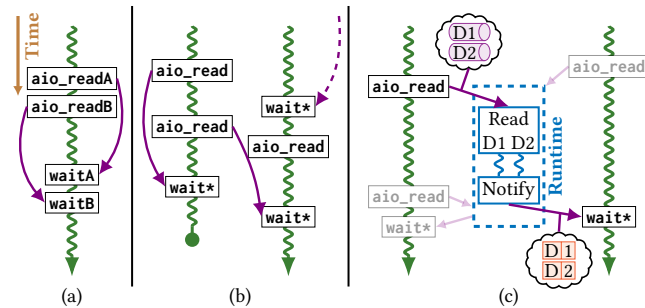


Figure 1. Decoupling GPU threads between initiation and consumption. (a) Case 1: Same thread waits for the specific I/O (A,B) it initiated. (b) Case 2: The thread waits for any I/O (*). (c) Case 3: Thread waits for correlated I/Os (D1+D2) with runtime ensuring combined delivery to consumer.

2.3 Case 3: Multiple Correlated I/Os

In applications such as Single-Source Shortest Path (SSSP), processing a parent vertex requires two correlated I/O operations - one for reading the edge of that parent (which is one

I/O operation - denoted D1 in Figure 1 (c)) and another for the weights of those edges (another I/O operation denoted D2 in Figure 1 (c)). Unlike previous cases where I/O operations were independent, here the (same) consuming thread needs results of both I/Os - the edges and weights - to proceed. Else, in a decoupled implementation as in the previous example, one thread could end up with the weight and another with the edge, necessitating additional synchronization and communication to exchange information. Instead, we want the same thread to consume both of these correlated I/Os.

To handle such dependencies, we introduce application-level grouping of I/O operations. The initiator can ask for multiple I/Os to be initiated in the same call by passing an array of control blocks (for D1 and D2). This informs the runtime of the correlations between these I/Os, which may then initiate each of these I/Os and wait for all their completions before notifying the “selected” consumer. The `g_wait()` returns a pointer to the results from these multiple I/Os, which is further detailed in Appendix A.2 for the SSSP application.

2.4 Summary of APIs

Table 2 lists the main APIs exposed to applications in our system. These can be invoked directly by GPU threads and are managed entirely by the GPU runtime (implemented as a megakernel [36] in CUDA code, explained in Section 3), thereby avoiding any CPU intervention. Before initiating the I/O, the thread(s) build the control block, along with the data structure pointed to by `app_ptr`, which is passed along in the control block when initiating the I/O. The thread(s) can consequently move on to any other computation or a new I/O issue, overlapping the initiated I/O with such useful work.

AGIO API	Description
<code>g_aio_read(cb)</code>	Initiate async. I/O read
<code>g_aio_write(cb)</code>	Initiate async. I/O write
<code>app_ptr = g_wait((cid, *))</code>	Wait for a specific I/O (cid)
<code>app_ptr = g_wait((*, tag))</code>	Wait for any I/O with matching tag
<code>app_ptr = g_wait((*, *))</code>	Wait for any I/O
<code>g_get/free_mem()</code>	(De)Allocate dynamic GPU memory
Note: the <code>app_data</code> pointer can serve as the <code>cid</code> (command id). Flags in <code>cb</code> (control block) can further control the I/O behavior.	

Table 2. AGIO API

Our API provides three ways to wait for I/O completion as shown in Table 2: (i) `g_wait((cid,*))` to wait for a specific I/O operation; (ii) `g_wait((*,tag))` to wait for any I/O initiated with the specified tag (similar in spirit to other split/asynchronous mechanisms (e.g. MPI [25]), tagging of I/Os by the initiator allows the threads to select/filter from a group of I/Os that they want to receive); and (iii) `g_wait((*,*))` to wait for any I/O operation to complete.

The application-specific data (`app_data`) passed on within the control block of an initiation is returned by these wait operations, in order to transmit any information from the initiator thread to the consumer thread.

3 Runtime System for AGIO

To perform microsecond level asynchronous I/O tasks entirely on current GPUs, without explicit hardware support, we next explore the software design choices.

3.1 Who should perform these tasks?

Given the hardware (mainly in-order execution of instructions) and ISA (which lacks non-blocking instructions) primarily designed for TLP (Thread-Level Parallelism), the only viable option for creating asynchronous tasks is to launch them as separate threads, which we term *Runtime Threads*, that run in parallel/background with the *Application Threads*. Below are two choices (due to SIMT, it does not make sense to mix application and runtime threads within the same warp) for launching and running these threads.

Warp Specialization [4]: Thirty-two GPU threads are grouped into *warps*, and each warp is executed by a GPU Streaming Multiprocessor (SM) in SIMT (Single Instruction Multiple Thread) fashion, time-slicing across multiple warps. We could create special warps containing runtime threads which wait for and perform the tasks given to them. They could be co-located with the warps of the application threads on the same SM, allowing better communication and state-sharing, if they are indeed matched up for such sharing.

SM Specialization: Although warp specialization could reduce communication costs, this is highly unlikely in practice. Due to workload and runtime variability, specialized warps may end up processing tasks for remote SM warps or being co-located with other specialized warps that have interfering working sets, leading to cache thrashing. Also, these specialized threads may need to coordinate/synchronize based on workload demands. Warp specialization imposes additional synchronization constraints/costs as noted in [46].

Further, kernel launch is managed at block granularity, with each block scheduled on an SM. This makes warp specialization difficult to implement using conventional techniques.

Consequently, we use SM specialization in our implementations. We use Nvidia’s recent Green Contexts [35] feature to physically partition the GPU at a granularity of four SMs (of the total 108 SMs on the A100) for apportioning between application and runtime threads. We implement AGIO’s runtime kernel (Figure 3) as a persistent megakernel [1, 6, 18, 36, 52] to reduce synchronization and launch overheads while maintaining better load balance. Based on empirical studies, we fix 32 SMs for the runtime threads with the remainder allocated to application threads. Later, we also present sensitivity results varying the SMs.

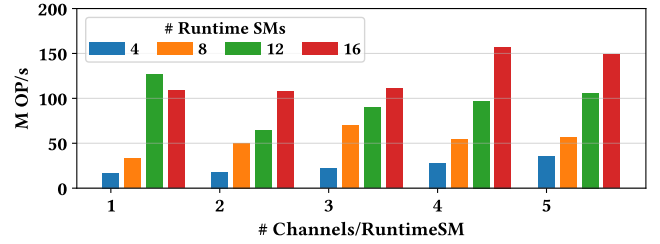


Figure 2. Queue performance with different # SMs.

3.2 How do we communicate with runtime threads?

Communication between application and runtime threads running on different SMs becomes a critical issue. Bi-directional communication is essential for: (i) application threads to submit I/O requests to the runtime threads; and (ii) runtime threads to notify application threads upon completing these requests. Both directions rely on multiple highly optimized queues designed to maximize concurrency.

Each channel requires two queues for bi-directional communication: request (what blocks/sizes need to be read/written and to/from where) and completion. A single channel can become a serialization bottleneck to numerous application and runtime threads. We conduct a throughput study varying the number of channels in Figure 2. With enough work being pumped in, the parallelism of multiple queues is more important, though the benefits start waning at large numbers. In fact, at 4 channels per SM, we see a throughput of more than 150M OP/sec, which far exceeds what one would get today even with multiple high-end SSDs (NVMe SSDs are only rated at around 1.5M IOPS [27, 38]). Hence, in our design, we set the number of channels at 4 per SM.

Highly concurrent queues/ring-buffers have been studied in different contexts [19, 26, 41]. For completeness, we borrow many ideas from such prior studies towards a more effective GPU implementation as discussed below.

Each queue consists of configurable, fixed-size slotted ring buffers with head and tail pointers. Apart from holding the control block (cb) explained in Section 2, each slot also has a 4-byte atomic state variable, to track whether it is valid/empty/processed. This variable is also multi-purposed to store a tag, or some specific commands. With multiple submission and wait queues, there arises the question of matching the queues to the threads. Based on several design choices that we investigated, we found that randomly selecting a queue to submit/wait is the best alternative. While theoretically this may lead to one or more threads waiting for work in one queue, with work to be done on another, given the high number of threads and parallelism in the workloads, this rarely happens.

Enqueue/Dequeue Operations: These operations need to be very efficient and highly parallel without serialization bottlenecks for scalability across numerous threads. To ensure maximum concurrency, we maintain two indexes to each

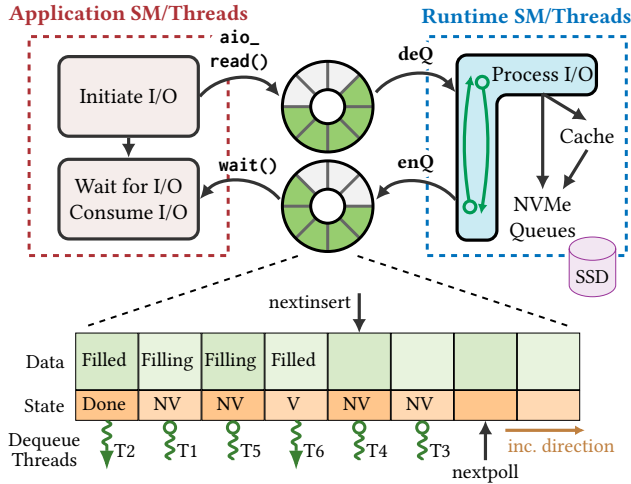


Figure 3. AGIO Runtime System. (i) Specialization: Application and Runtime threads run on their designated SMs. (ii) Queues: the `nextinsert` and `nextpoll` pointers atomically increment. The T2 and T6 dequeue threads continue processing their entries and may mark their slots as Done, despite “holes” (polled by T1, T5) in the queue. Meanwhile, other threads (T1, T5, T4, T3) poll their slot’s state, waiting for NotValid(NV) to transition to Valid(V).

circular queue: `nextinsert` (next position where an item is to be added) and `nextpoll` (last slot where threads are already waiting/polling for work to be removed).

To enqueue, threads first atomically increment `nextinsert` to find a slot for insertion. They can subsequently, at their leisure, add different fields into this slot while other threads are free to increment `nextinsert` and add their respective entries into their own slots. To dequeue, threads first atomically increment `nextpoll` to get their slot to remove. Note that this does not automatically imply that there is data to remove (yet to arrive) - it is only a placeholder. The thread can independently poll on that slot’s state (checking the “valid” flag in Figure 3). Other threads, can meanwhile poll on their own respective slots and/or remove from there concurrently.

Unlike CPU threads, which can be created and destroyed dynamically, GPU threads remain active throughout execution. Also, applications cannot directly control state saving/restoring of GPU threads and their scheduling. Consequently, to handle the producer-consumer problem of bounded buffers/queues, polling is the only option and our design tries to provide maximum concurrency.

Our `nextinsert` and `nextpoll` are not necessarily analogous to the typical head/tail variables used in queues. They are what may be viewed as lazy implementations of these variables, rather than exact implementations. `nextinsert` just indicates the placeholder for the next thread that is trying to add, rather than pointing to the tail of the queue (it may not yet have been inserted). Further, if more threads atomically increment `nextinsert`, they may even succeed in

adding elements to the queue before prior threads fill an earlier slot, i.e. we may have holes momentarily. Similarly, while `nextpoll` is a placeholder for threads to wait for elements to appear, they may again not be in strict FIFO order, i.e. a thread which comes later may find a valid entry to remove from its slot while earlier threads may still be waiting.

This approach offers several advantages: (i) allows for “holes” in the queue, relaxing strict ordering constraints and allowing neighboring threads to proceed; (ii) the enqueue thread only increments `nextinsert` and does not need to worry about other variables; and (iii) there is no doorbell-style notification mechanism [17], which may become a serialization point. The trade-off is the need for an atomic variable per queue slot, but this overhead is minimal given the high parallelism and substantial bandwidth of the underlying hardware (HBM), making it particularly advantageous in GPUs.

An application thread invoking `g_aio_read/write()` adds the I/O operation to the submission queue, and a runtime thread polls on this queue to pick up work. After I/O completion, the reverse happens, and an application thread calling `g_wait()` will eventually poll and get the I/O results. There are additional details (tags, dedicated state cleanup warp, etc.) not explained due to space constraints.

3.3 Memory Pool and Allocator

Dynamic “in-kernel” GPU memory allocation is critical to support our mechanisms. Most GPU applications avail of pre-allocated (by the CPU) GPU memory during their execution. We use the latter (i.e. CPU) only once for the overall pool allocation, which spans the entire GPU memory on the card to pre-allocate pools of different sizes (e.g. 32~16K bytes). Our runtime rations this out via our `g_get/free_mem()`.

Upon a memory request, the queue with the most suitable slot size is selected and its head pointer is atomically advanced. Similar to queues in Section 3.2, each slot includes an atomic status variable indicating whether it is in use or free. If in use, the allocator continues advancing the head until a free slot. For requests exceeding the largest pre-allocated sizes (e.g. 16K), the allocator increments the head by a multiple of the slot size to ensure atomic allocation and contiguous memory blocks. Additionally, when required, the allocator manages the translation of GPU virtual addresses to DMA (I/O) addresses required by the NVMe device to transfer data back-and-forth between I/O device and GPU memory.

Further optimizations for in-kernel, highly parallel GPU memory allocators remains an open area for future work.

3.4 Caching

A prior work, BaM [37], implements memory-mapped I/O between GPU memory and NVMe SSDs (not asynchronous). BaM incorporates a caching layer to avoid repeatedly going to SSDs. We simply build upon this caching layer to incorporate our asynchronous I/O mechanism. Since BaM uses

memory-mapped I/O, while AGIO is explicit I/O, we do incur an additional level of copying between underlying system buffers and application data structures.

As with BaM, AGIO interfaces with the SSDs via NVMe queues allocated in GPU memory (initially mapped by the CPU through PCIe BAR). Because this memory region supports PCIe P2P, GPU threads can directly transfer data to and from the NVMe device without CPU intervention or bounce buffers. Consequently, AGIO does not have the layers found in the Linux I/O stack (e.g. the block layer, `io_uring` [3]).

4 Evaluation

We have implemented AGIO that involves a combination of (i) application enhancements to leverage asynchrony (decoupling initiation from completion, with useful computation in between when possible); (ii) APIs for applications to avail of AGIO calls; and (iii) a runtime implemented in CUDA code running on GPU threads (which wait for I/O requests from application threads, issue the requisite I/O calls, and subsequently notify application threads upon I/O completions). We have evaluated this on a Nvidia A100 GPU installed on a server system whose configuration is listed in Table 3.

System	Tyan B7119F83V8E4HR-2T-N
CPU	Intel Xeon Gold 6226R x2
GPU / SSD	Nvidia A100 40GB PCIe / Micron 7450
Software	Linux 6.8, Nvidia 570.124.06, CUDA 12.8

Table 3. System Specifications

4.1 Microbenchmark

It is important to understand “when” and “why” the non-blocking AGIO would be preferable over its synchronous I/O counterpart (where threads issue and subsequently block for completion). This will better explain the complete application execution results later. Towards this goal, we built a microbenchmark to evaluate the impact of key parameters affecting the relative benefits - (i) the number of threads requiring I/O (which we will denote by p , the parallelism factor); and (ii) the amount of computation (compute intensity) that a thread can perform between issuance and completion of I/O (which we denote by o , the overlap factor calculated in FLOPs/byte of I/O). The microbenchmark is straightforward in design. In the asynchronous version, a set of p threads repeatedly initiate I/O, retrieve any I/O that has already completed (potentially initiated by other threads), and perform the computation o decided by the I/O size. In the synchronous version, each thread performs its computation only after waiting for the completion of its own I/O request.

For different p values (x-axis) in this benchmark, Figure 4 shows (a) speedup of AGIO over the synchronous version; and (b) I/O requests/sec on average during the execution. These are shown for different amounts of overlap o . From

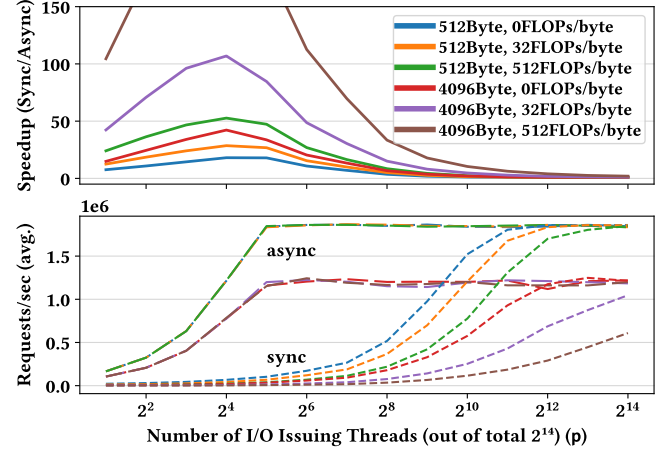


Figure 4. Microbenchmark. (a) Speedup of AGIO vs. Synchronous for different (I/O size, o); (b) I/O Issue Rates: Long dashes are for AGIO and short dashes for Synchronous.

these graphs, we note: (i) having more overlap o with ongoing I/O helps boost AGIO speedups, as expected. This leverages idle GPU cycles towards useful computation, rather than simply waiting for I/O; (ii) the number of threads initiating I/O, p , shows interesting trade-offs. At small values, the underlying system I/O bandwidth is underutilized and any computation overlap is relatively small for low p (both AGIO and the synchronous versions have low number of requested I/Os in Figure 4 (b)). At the other end, when p is very large, performance is limited by the I/O bandwidth (we see pending requests flattening out to the extreme right at similar values for AGIO and synchronous versions). There is a relatively large region of p values (between 4–512) where AGIO significantly outperforms the synchronous version. In this region, the system has sufficient I/O bandwidth, but the synchronous version is unable to initiate sufficient I/Os to leverage it - even threads not incurring I/O currently cannot proceed to initiate future I/Os, leave alone the inability to overlap compute. We see this effect in Figure 4 (b), where AGIO threads are able to initiate I/Os much faster.

Though intuitive in retrospect, the key takeaways are: (i) AGIO requires sufficient computation to be overlapped (o) between I/O initiation and completion; and (ii) I/O imbalance (i.e. not all threads are I/O free or all I/O issuing) helps AGIO significantly by allowing (non-issuing) threads to proceed faster to initiate future I/Os that can all be served in parallel to more fully utilize the underlying bandwidth.

4.2 Workloads

As the above results show, data accesses (p , determines whether needing I/O and when) and its usage (o , overlap with I/O) have an important determination on AGIO’s effectiveness. At the same time, AGIO requires applications to be programmed explicitly with I/O initiation and completion APIs (Section 2). p and o for a real application will depend on

Type	App	Config./Input	Use Cache?	FLOPs/IO
static	gemv(mv)-4K	IO Size: 4KiB	No	Low
	gemv(mv)-1M	IO Size: 1MiB	No	High
	kmeans(km)-8	Centroids: 8	No	Low
	kmeans(km)-128	Centroids: 128	No	High
dynamic	bfs	k* / u*	Yes	Low
	sssp	k* / u*	Yes	Low
	pagerank(pr)	k* / u*	Yes	Low
	cc	k* / u*	Yes	Low

(a) Application Characteristics

Dataset	# Vertices	# Edges	Deg.(avg/std)	Size
k16	134M	1.07B	15.84 / 364.39	32GiB
k32	134M	2.11B	31.47 / 695.01	64GiB
k48	134M	3.15B	46.94 / 1007.61	94GiB
k128	134M	8.24B	122.78 / 2420.35	246GiB
k192	134M	12.2B	182.10 / 3443.04	366GiB
u16	134M	1.07B	16 / 4.00	32GiB
u32	134M	2.15B	32 / 5.66	64GiB
u48	134M	3.22B	48 / 6.93	96GiB
u128	134M	8.59B	128 / 11.31	256GiB
u192	134M	12.88B	192 / 13.86	384GiB

(b) Datasets ¹ (for dynamic apps)**Table 4.** Workloads

how effectively the programmer/compiler can insert these calls. This, in turn, largely depends on the static (compile-time with some regularity) vs. the dynamic (runtime with randomness) nature of the application, i.e. when in the execution would we be able to determine what data to access and hide this latency. Taking these factors into consideration, our workload suite includes representative static and dynamic applications as shown in Table 4.

gemv and kmeans are, from our viewpoint, static applications, where at programming/compile time, we know what data will be needed and when, to appropriately insert I/O initiation/completion calls, optimizing for the p and o factors - similar in spirit to compiler/programmer based prefetching in regular applications. gemv is a dense matrix-vector product kernel, where we assume the matrix is on the SSD. The programmer knows exactly what data will be needed and when, and can thus insert I/O initiations much ahead (like prefetching), and overlap the product computation for already fetched data, before waiting for new data. There is little reuse, and since all threads need data from SSD in a fairly synchronized manner, nearly all threads will be initiating I/O requests at the same time (i.e. relatively high p). As far as o is concerned, we could have different granularities of initiating I/O, with consequent high or low computation on such data being overlapped with the I/O operation. We consider two different request sizes - 4 KiB and 1 MiB - leading to very different amounts of computation possibly overlapped

with each I/O operation² (we will consider a wider spectrum in later experiments of Section 4.4). kmeans [12] is another with regular access patterns known a priori, that can be accordingly programmed to maximize the overlap. In this case, we use two centroid values (8 and 512) to control the amount of computation that is overlapped per I/O initiation.

Graphs and pointer-chasing applications fall in the dynamic (accesses known only at runtime [9]) category. In such cases, threads can perform I/O initiations only when they know what data they will need, e.g. BFS needs pointers from the current level to fetch nodes for the next level. We use 4 applications in this category - BFS, SSSP, PR, and CC - where the I/Os have relatively little slack on when they can be initiated (as in BFS, only when the pointer information is obtained and the computations on the data fetched are rather immediately after). This limits the amount of overlap (o) in these applications, with the programmer having little flexibility in increasing the overlap. However, the input dataset/graphs can have a significant impact on the p factor, and thereby AGIO's effectiveness. For instance, a low degree graph may not have much parallelism during traversals (i.e. low p), while a high degree can allow a larger p (even if o is low). To consider these effects, in these dynamic graph applications, we use a spectrum of input graphs listed in Table 4, with diversity in their node degrees and their variance. Another facet of these graph applications is the possibility of reuse, since during the traversals, a thread may revisit an edge/vertex - thus being influenced by the effect of the cache (which will indirectly impact p as well).

4.3 Comparison with Synchronous I/O Baselines

There are several mechanisms to avail of "synchronous" I/O from GPU threads, with most of them being orchestrated by the host CPU as mentioned earlier. However, recent work [37] has shown that the CPU becomes a bottleneck for high throughput transfers required by GPU threads, and instead proposes fully GPU-orchestrated transfers between SSDs and GPU memory. Hence, rather than comparing with all prior synchronous I/O mechanisms, we primarily compare with the following state-of-the-art mechanisms that have been shown to perform better: (i) *BaM-baseline* [37] provides an `mmap()`-like API for GPU kernels which maps the SSD and faults on non-resident SSD blocks. The subsequent fault handler is directly implemented by GPU threads (rather than host CPU drivers), which program the NVMe queues to perform data transfers between SSDs and GPU

¹ Generated using [5], where k^* is based on kron (high degree variance) and u^* is based on urand (low degree variance). The $*$ denotes the avg. degree.

² Even though FLOPs/byte of I/O is the same across gemv-4K and gemv-1M, we are using FLOPs/IO here as an indication of the o factor. This is because, it is usually much faster to transfer a large volume of bytes per I/O rather than several short individual transfers. Hence, with 1 MiB transfers, we are incurring lower I/O time per byte, allowing better overlap with computation to hide this I/O.

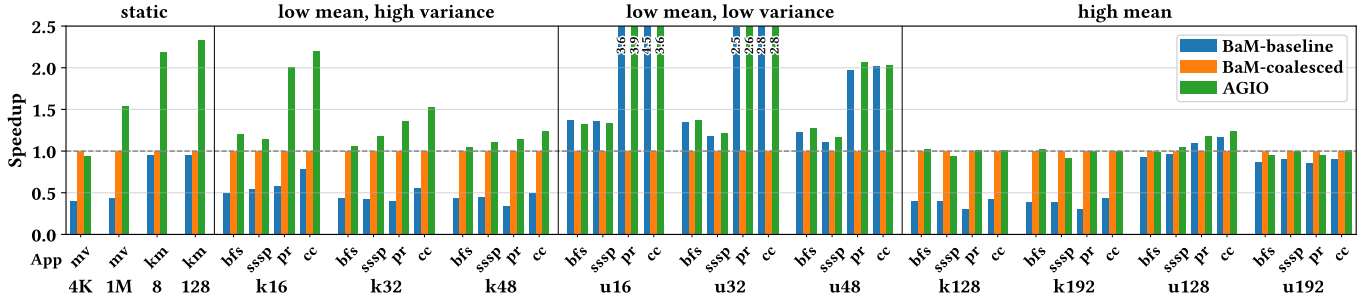


Figure 5. Speedup over BaM-coalesced.

memory. It also implements a cache to avail of any locality behaviors. (ii) *BaM-coalesced* is an enhancement over BaM-baseline, where threads within a warp are largely assigned to successive I/O offsets (spatial locality) - allowing a higher possibility for the requests to be coalesced [29] before accessing the cache or issuing NVMe requests to SSDs.

It is important to understand how these two assign work to threads in a warp. BaM-baseline does not look for I/O coalescing opportunities within a warp. For instance, in static applications like *gemv*, it may assign separate rows of the matrix to different threads of a warp while BaM-coalesced shall assign successive portions of a row to threads of a warp to coalesce their I/O requests (considering the data layout on the SSD). Similarly, in graph applications, BaM-baseline may assign a thread to a block of pointers for traversal one after another, as opposed to splitting this block amongst threads of a warp to traverse as in BaM-coalesced. As we will see below, these assignments can have an impact on the load balance that gets compounded by I/O overheads depending on the nature of input datasets.

Figure 5 shows the speedups of AGIO vs. the state-of-the-art BaM, both normalized with respect to the performance of BaM-coalesced, which is expected to perform better than BaM-baseline. Before getting into detailed analysis, we note the overall performance benefits of AGIO over BaM-coalesced - (i) 1.65x speedup for the static applications; and (ii) 1.32x speedup for the dynamic applications on average. We next explain the relative performance across applications using the p and o factors explained above.

Static Applications: In this category, all four executions (*gemv*-4K, *gemv*-1M, *kmeans*-8, *kmeans*-128) have high p values, because they are programmatically load balanced across the threads, and nearly all threads will hit the I/O initiations at similar times, especially since these are all first-time accesses (and uncached). As a result, we are operating in the extreme right region of Figure 4 (b) in terms of p , implying that AGIO may not enjoy the benefits of issuing I/O requests faster. With respect to the o factor, we see that *gemv*-1M and *kmeans*-128 have much higher o , and thus much higher speedups (1.54x and 2.33x) versus their lower

o counterparts, *gemv*-4K and *kmeans*-8 (0.93x and 2.18x) over BaM-coalesced. Even between *gemv* and *kmeans*, the latter does more compute (higher o) for the data fetched from SSDs, thus providing higher speedups. In summary, for the static applications, the hand-tuned higher o offsets the high p factor, to still provide the desired benefits for AGIO.

Dynamic Applications: In this category, the programmer has little control, and the o and p factors are largely determined by the input datasets (Table 4). All are graph/pointer chasing applications, retrieving data from SSD based on those pointers, with little computation performed. Hence, irrespective of the graphs, the o factor is quite low in these applications. Therefore, the p factor plays a more important role in AGIO benefits in this case. From an application characterization viewpoint, one can get an intuitive sense of the relative parallelism p of I/O initiation by GPU threads as they traverse the input graphs if we examine their average degree/fan-out (number of edges per vertex).

Low Mean, High Degree Variance (k16, k32, k48): High variance leads to some graph nodes having a lot of work (children), with some others having much less or even zero children. In such graphs, BaM-baseline may have a lot of work imbalance across threads within a warp - since a thread assigned to a vertex with many children (high degree) has a lot more work to do compared to a thread of the same warp assigned to a vertex of low degree. This work imbalance makes BaM-baseline perform much worse (0.49x on average compared to BaM-coalesced) for these graphs (Figure 5).

For the same graphs, BaM-coalesced which assigns all threads within a warp for a vertex, can also experience work imbalance across the warps - some warps may have all threads assigned edges/work, while some others assigned to a vertex with low degree may not utilize all threads of the warp. While this effect is less pronounced than the load imbalance of BaM-baseline within a warp, it is still substantial for these high variance (in the degree) graphs, causing its performance to be worse than our AGIO execution. AGIO provides 1.31x speedup over BaM-coalesced in these graphs.

In contrast, AGIO is less impacted by any such load imbalance, and is less sensitive to the work assignment itself.

Whenever threads, within a warp or across warps, run out of any work that requires I/O initiation, they can simply (non-blocking) go ahead and process other work (waiting and consuming other I/O, or exit early). This is an important additional benefit of AGIO, i.e. it is relatively immune to the nature/characteristics of workloads, the data layout, and work assignment, unlike BaM-baseline or BaM-coalesced. We see evidence of this in Figure 6, where AGIO’s non-blocking threads are able to proceed and issue I/O requests much faster, and thus finish earlier, compared to the other two mechanisms.

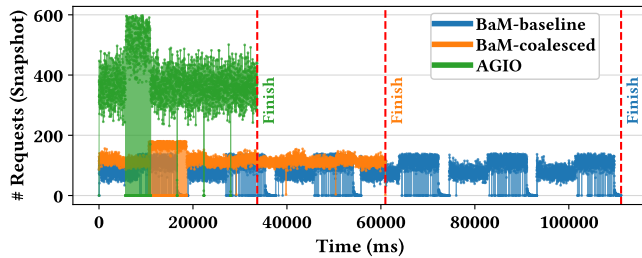


Figure 6. Number of requests in NVMe layer taken at 1ms snapshot interval in k16-pagerank(pr). AGIO issues requests much faster to finish earlier than others.

Low Mean, Low Degree Variance (u16, u32, u48): For low degree graphs, BaM-coalesced may not have work for all threads in a warp, especially for say the u16 graph (since the warp size is 32). On the other hand, in BaM-baseline, since the variance in node degree is low, the load will be relatively balanced across warps and the warps themselves will have work assigned for all their threads. As a result, BaM-baseline does significantly better (1.87x) than BaM-coalesced, especially in the u16 graph where the latter’s warps are highly under-utilized. At the other end, while u48 has a higher degree, note that BaM-coalesced will employ multiple iterations of the threads in a warp to process more than the 32 edges (warp size of 32), still resulting in load imbalance across these threads. While one may think u32 will not suffer such problems, there is still a non-zero variance in degree across nodes, leading to work imbalance in some nodes.

Even in this case, we notice that AGIO is agile enough to adapt to the graph characteristics. Regardless of the work to thread and warp assignment, threads do not block when initiating I/O and are able to proceed further ahead to initiate more I/O. Further, we are decoupling the initiation from the wait/consumption. This allows any thread to consume the results of an earlier initiated I/O, thus negating much of the load imbalance effects. As a result, AGIO does significantly better (1.87x) than BaM-coalesced, and does as well as BaM-baseline in this case. Note that the latter has taken advantage of a priori knowledge of graph characteristics towards work assignment, while AGIO is oblivious to the nature of the graphs and adapts itself dynamically.

High Degree Mean (k128, k192, u128, u192): These graphs have high node degrees (mean of 128 and 192), with the first two graphs having high variance while the latter two have low variance. As noted in previous results, load imbalance within a warp arising from low mean and/or high variance in the node degrees, impacts the performance of BaM-coalesced making it perform worse than BaM-baseline. When the node degree is high, this load imbalance impact decreases, and as a result, whether there is high or low variance, we see BaM-coalesced starting to do better - does even better with low variance graphs. Again, we see that AGIO does well across the spectrum, and in many cases outperforming the better of BaM-baseline or BaM-coalesced.

Summary of AGIO benefits: A key insight from the above results is the resilience of AGIO to the dataset characteristics. It is less sensitive to work assignments to threads of a warp, with the decoupling of initiation from wait/completion allowing it to dynamically adapt to the graph properties in these workloads. This makes it a more compelling mechanism, especially with insufficient a priori details of input datasets - graph topologies, cache behavior, computation for each vertex, probability of accessing neighbors, etc.

These results point to the agility of AGIO to benefit from (i) high computation overlap (o) when available; (ii) allowing subsets of threads to proceed even if some are stalled for I/O; (iii) allowing such threads to progress towards issuing more I/O requests, to avail of high bandwidths; and (iv) the consequent higher utilization and lower work imbalance despite the vagaries of input data without programmer involvement.

4.4 Comparison with Other Asynchronous Baselines

Having compared AGIO with state-of-the-art synchronous I/O mechanisms, we next compare it with alternatives for asynchronous I/O. One can mimic such behavior from the CPU using CUDA streams [30] which allow launching multiple GPU kernels, to subsequently execute in an asynchronous manner. This would allow launching separate asynchronous routines for deterministic and a priori known access patterns, and we use the GDS (cuFileAsync [32]) interface to initiate the I/O from the SSD to the GPU.

Note that this is more easily done for *static* applications, since there is a priori knowledge of data accesses (i.e. upon kernel launch from the CPU). It is not just cumbersome, but can also be inefficient for dynamic applications where this is available only at runtime. In Figure 7, we show results with this technique for gemv for different I/O size granularities (impacting *o* in AGIO) that are asynchronously read in advance, while computing on already retrieved data.

Across the spectrum of I/O sizes, AGIO does much better than trying to mimic such behavior with streams on the CPU due to two factors: overheads of streams which work

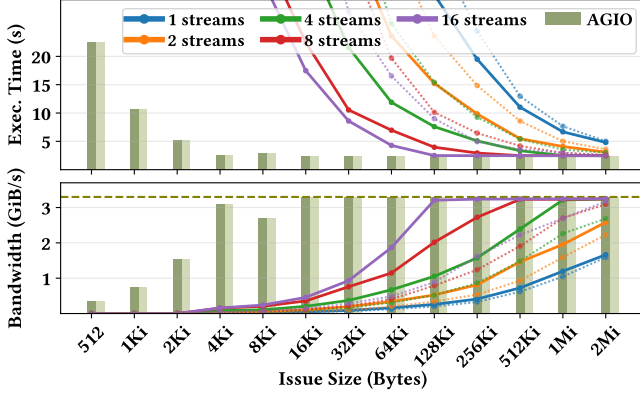


Figure 7. gemv with AGIO (bars) and multi-stream GDS/cuFileAsync (lines). Darker bars and solid lines are without extra load on the host CPU, while lighter bars and and dotted lines are with sysbench running on host CPU.

at kernel granularity as opposed to threads in AGIO, and CPU orchestration in GDS vs. direct GPU orchestration in AGIO. Though not saturating I/O bandwidths, at low I/O granularities, AGIO initiates I/O requests at a much faster rate to provide the necessary speedups. Once we go beyond 8K granularities (GDS needs to employ 128K granularities or larger), AGIO is able to saturate the underlying system bandwidth of 3.3GiB/s.

We note two facets of AGIO in these experiments: (i) GDS approach involves the host CPU in kernel launches as well as in initiation (cuFileAsync) of data transfers. To some extent, we are assuming the CPU is entirely at its disposal for these tasks. We also run experiments where the CPU is busy by running “sysbench” [42]. As can be seen, AGIO is hardly affected by the load on the CPU, making it much more preferable than the GDS approach (dotted lines) where there is considerable deterioration due to the additional CPU load; (ii) our AGIO approach fulfills the intentions of “application-directed prefetching” which have been extensively studied for CPU caches and I/O. A purely runtime application agnostic prefetching mechanism can lead to useless prefetches, early evictions, and interference with normal traffic. In this experiment, the programmer has used AGIO for explicit initiation, like a prefetcher (when and how much that has been varied in the x-axis of Figure 7), to control such factors. In this case, issuing it early enough at a granularity of 8K or larger seems to be as good as more sophisticated choices. However, future work is needed for a more in-depth study of when and how much to initiate for a wider set of applications.

4.5 AGIO for better utilization?

Given that I/O can result in GPU underutilization, one is interested in whether we can get the same overall performance with a smaller GPU using AGIO. To investigate this, we limit the number of hardware SMs on Nvidia A100 GPU by using

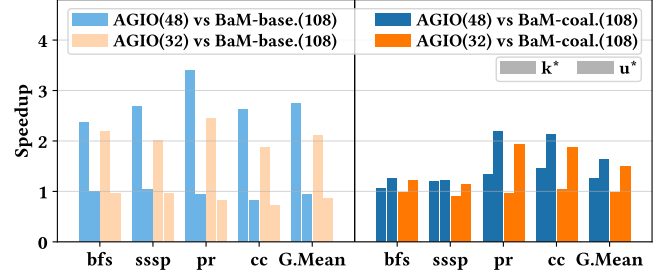


Figure 8. Speedup of AGIO with reduced number of hardware SMs (48/32 vs 108). The left lighter group compares with BaM-baseline and the right darker group compares with BaM-coalesced, both with full (108) amount of SMs.

Green Contexts [35], and conduct experiments with the full 108 SMs, 48 SMs, and 32 SMs.

Figure 8 compares AGIO execution on a smaller GPU with 48 and 32 SMs, with BaM-baseline and BaM-coalesced running on the entire 108 SMs. We show this for both kinds of graphs (high and low degree variances) for each of the graph workloads, along with the Geometric Mean. Overall, we see that AGIO is able to provide nearly as good performance with a much smaller GPU (48 and 32 SMs) compared to BaM-baseline and BaM-coalesced running on the entire 108 SM GPU. In fact, in the k* (high degree variance) graphs, it does almost twice as good in many cases, showing how well it can utilize a much smaller GPU’s resources compared to the other two mechanisms. Even in the earlier experiments, AGIO was significantly outperforming the other two on the 108 SM system. In the u* graphs, while AGIO does significantly better than BaM-coalesced (as in earlier experiments), outperforming the 108 SM system even on a much smaller GPU, the difference between AGIO and BaM-baseline is not very significant for these graphs despite more than halving the size of the GPU in the former.

These results demonstrate AGIO’s ability to dynamically adapt and utilize GPU resources more effectively during I/O, much better than the alternatives. With higher I/O intensities, not blocking upon I/O and decoupling initiation from wait/consumption will make AGIO an even better choice.

5 Related Work

With the need to extend the reach of GPU threads to data residing on host memory and storage, recent work in this direction can be categorized into the following.

CPU-orchestrated: Numerous vendor offerings (e.g. Nvidia UVM [31], AMD Unified Memory [2], HMM [22]) and research proposals (SPIN [7], Dragon [23], ActivePointers [39], [8, 11, 13–16, 20, 43, 47]) address data movements between host, GPU memory, and other devices, with many of the data transfers going through DMA engines. This has also been extended to storage devices/SSDs [28, 32, 40, 44, 49–51]. However, these require the host CPU to initiate/orchestrate

the transfers, which has been shown to become a bottleneck for the intended purposes.

GPU-orchestrated: Recent proposals (BaM [37], libnvm [24], GMT [10]) enable direct GPU orchestration of data transfers between its memory and SSDs. However, these mechanisms are synchronous, with threads blocking till the data transfer is completed, leading to underutilization as shown.

Asynchrony in Data Transfers: GPUs largely rely on a synchronous programming and execution model, and there is little hardware support for asynchronous execution. At best, there are a few mechanisms [33, 34, 45] to enable asynchronous transfers between on-card repositories (e.g. GPU global memory and caches). CUDA streams could be employed (in conjunction with GDS [32]) to achieve asynchronous I/O, but as our results show, it is quite inefficient and also not very general purpose.

Asynchronous explicit I/O from GPU threads is little explored, which is the gap that AGIO fills. A recent work [48] proposes similar asynchronous I/O mechanisms. While both systems pursue asynchrony, they differ in scope and emphasis. AGIO (i) focuses on providing a programming model that supports diverse use cases for decoupling initiation from completion; (ii) examines designs such as scheduling (specialization), queues (for maximizing concurrency without blocking), and coalescing; and (iii) evaluates using a comprehensive suite of dynamic (beyond static) applications with varied dataset properties to explain when and why asynchrony is useful.

6 Conclusions and Future Work

We present the design, implementation and performance benefits of AGIO, which provides an API and runtime environment to support asynchronous I/O directly from GPU threads. By decoupling, in time and space, I/O initiation from completion and subsequent usage, AGIO can (i) hide microsecond level SSD latencies whenever applications allow sufficient computation overlap; (ii) allow threads to proceed beyond the immediate I/O initiation, towards initiating more I/O that can effectively utilize the underlying system bandwidth; (iii) enable better load balance across threads of a warp and across warps by becoming less sensitive to I/O stalls that may not be fully balanced across threads; (iv) provide better utilization, with even half the number of SMs providing the performance (or better) of a full-fledged GPU; and (v) make the execution dynamically resilient/adaptive to dataset nuances without requiring programmer intervention.

There are several future directions to enhance AGIO including more efficient runtime modules (e.g. memory allocators, dynamic SM allocations between application and runtime threads), hardware support to make some of these software implementations more efficient (hardware notifications instead of software polling, hardware queues, etc.), algorithmic optimizations in order to leverage asynchrony,

as well as integration with other system/server components (multi-GPU systems, network interfaces, remote storage, CXL, etc.). AGIO is a starting point towards a more extensive set of asynchronous invocations to many high latency tasks and multi-device spanning workloads (e.g. RAG pipelines, Mixture-of-Experts models, etc.) that we intend to explore.

Acknowledgments

We would like to express our special thanks to the anonymous reviewers for their constructive feedback and suggestions. This work is supported in part by NSF grants 2211018, 1909004, and 1763681.

A Appendix

A.1 AGIO listing for BFS (Different Threads for Initiation and Completion)

```

1  /* Metadata to be transferred from
2     I/O initiator to I/O consumer */
3  struct app_data {
4     uint64_t *child_ptr;
5     uint64_t parent_id; uint32_t n_child; ...
6  };
7  /* I/O initiating thread: read children */
8  __device__ void build_bfs_cb(ctrl_blk *cb) {
9     uint64_t my_vertex = tid;
10    cb->offset = ...; cb->n_bytes = ...;
11    cb->buf = g_get_mem(cb->n_bytes);
12    app_data *ad = g_get_mem(sizeof(app_data));
13    ad->child_ptr = cb->buf; ad->... //Fill ad
14    cb->app_ptr = ad;
15  }
16  /* I/O consuming thread: mark visited */
17  __device__ void work_bfs(app_data *ad) {
18     for (i = 0; i < ad->n_child; ++i) {
19         uint64_t vid = ad->child_ptr[i];
20         if (vid not visited) mark visited;
21     }
22     g_free_mem(ad->child_ptr);
23     g_free_mem(ad);
24  }
25  /* Main kernel (entrypoint of all app threads) */
26  __global__ void main_kernel(...) {
27     ctrl_blk cb;
28     build_bfs_cb(&cb); //Build cb to read children
29     while (true) {
30         g_aio_read(&cb); //Issue to AGIO runtime
31         app_data *ad = g_wait((*,*));
32         if (ad) work_bfs(ad);
33         if (cb.issued == true) break;
34     }
35  }

```

Listing 2. BFS with AGIO

In this BFS example, the I/O initiator thread allocates (Line 28) application-specific data that should be transferred to the I/O consumer (id of the parent, the number of children,

a pointer to an array of child vertices - the I/O buffer, etc.), all grouped together in the `app_data` structure (Line 13). It then sets the `app_data` pointer in `cb` (Listing 1) to point to this newly allocated structure (Line 14). This `cb` is then used to initiate the I/O (Line 30).

In Line 31, a thread waits for I/O completion, not worrying about which I/O or who initiated it, by specifying `(*,*)` for `g_wait()` (Table 2). The I/O result is copied by the AGIO runtime into the buffer specified in `cb`, and the `g_wait()` API returns the `app_data` field of the control block for which the I/O has completed. The consumer thread can then use this pointer to process the data returned from the I/O (Lines 18-21) and then frees the dynamically allocated GPU global memory for this I/O when done (Line 22-23).

A.2 AGIO listing for SSSP (Multiple Correlated I/Os)

```

1 struct app_data {
2     uint64_t *edge_ptr, *weight_ptr;
3     uint64_t parent_vid, uint32_t n_child, ...
4 }
5 __device__ void build_sssp_cb(ctrl_blk *cb) {
6     uint64_t my_vertex = tid;
7     cb[0].offset = ...; cb[0].n_bytes = ...;
8     cb[0].buf = g_get_mem(...); //buf for edge
9     cb[1].offset = ...; cb[1].n_bytes = ...;
10    cb[1].buf = g_get_mem(...); //buf for weight
11    app_data *ad = g_get_mem(sizeof(app_data));
12    ad->... //Fill ad (incl. buf for edge & weight)
13    cb[0].app_ptr = ad; ...
14 }
15 __device__ void work_sssp(app_data *ad) {
16     //...
17 }
18 __global__ void main_kernel() {
19     ctrl_blk cb[2]; //cb for two I/Os
20     build_sssp_cb(cb); //Build correlated
21     while (true) {
22         g_aio_read_n(cb, 2); //Issue correlated
23         app_data *ad = g_wait(*,*);
24         if (ad) work_sssp(ad);
25         if (cb[0].issued == true) break;
26     }
27 }

```

Listing 3. Single-Source Shortest Path (SSSP) with AGIO

In this example, the I/O initiator thread is issuing two (Line 22) I/Os - one for edge I/O and another for weight I/O (line 8, 10). The two I/Os are correlated, thus they are useless if not consumed together. AGIO provides a variant API `g_aio_read_n(cb, n_cb)` (Line 22) which guarantees all `cbs` issued together will be complete on return from `g_wait()`.

After issuing both I/Os through this single API function, the thread becomes an I/O consumer thread and waits for any I/O (Line 23). This `g_wait()` returns the `app_data` pointer

of the I/O, containing both edge and weight data (Line 2) upon return. Afterwards, it can consume and further process the data (i.e. perform the SSSP algorithm on the read data) (Line 24).

B Artifact Appendix

B.1 Abstract

We provide the source code for AGIO.

B.2 Artifact check-list (meta-information)

- Publicly available?: Yes.
- Archived (provide DOI)??: Yes.

B.3 Description

B.3.1 How to access. The source code of AGIO is accessible at <https://doi.org/10.5281/zenodo.18333270>

B.3.2 Hardware dependencies. Nvidia A100 GPU, Micron 7450 NVMe SSD

B.3.3 Software dependencies. Linux 6.8, Nvidia driver 570, CMake 3.31, CUDA 12.8, GCC 12

B.4 Installation

Please refer to the `README.md` in the source code for additional details.

References

- [1] Timo Aila and Samuli Laine. 2009. Understanding the efficiency of ray traversal on GPUs. In *Proceedings of the Conference on High Performance Graphics 2009* (New Orleans, Louisiana) (HPG '09). Association for Computing Machinery, New York, NY, USA, 145–149. doi:10.1145/1572769.1572792
- [2] AMD. . *Unified memory*. https://rocm.docs.amd.com/projects/HIP/en/docs-6.2.0/how-to/unified_memory.html
- [3] Jens Axboe. 2019. *Efficient IO with io_uring*. https://kernel.dk/io_uring.pdf
- [4] Michael Bauer, Henry Cook, and Bruce Khailany. 2011. CudaDMA: optimizing GPU memory bandwidth via warp specialization. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis* (Seattle, Washington) (SC '11). Association for Computing Machinery, New York, NY, USA, Article 12, 11 pages. doi:10.1145/2063384.2063400
- [5] Scott Beamer, Krste Asanović, and David Patterson. 2017. The GAP Benchmark Suite. arXiv:1508.03619 [cs.DC] <https://arxiv.org/abs/1508.03619>
- [6] Mehmet E. Belviranlı, Seyong Lee, Jeffrey S. Vetter, and Laxmi N. Bhuyan. 2018. Juggler: a dependence-aware task-based execution framework for GPUs. In *Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Vienna, Austria) (PPoPP '18). Association for Computing Machinery, New York, NY, USA, 54–67. doi:10.1145/3178487.3178492
- [7] Shai Bergman, Tanya Brokhman, Tzachi Cohen, and Mark Silberstein. 2019. SPIN: Seamless Operating System Integration of Peer-to-Peer DMA Between SSDs and GPUs. *ACM Trans. Comput. Syst.* 36, 2, Article 5 (apr 2019), 26 pages. doi:10.1145/3309987
- [8] Tanya Brokhman, Pavel Lifshits, and Mark Silberstein. 2019. GAIA: An OS Page Cache for Heterogeneous Systems. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. USENIX Association, Renton,

- WA, 661–674. <https://www.usenix.org/conference/atc19/presentation/brokhman>
- [9] Martin Burtcher, Rupesh Nasre, and Keshav Pingali. 2012. A quantitative study of irregular programs on GPUs. In *2012 IEEE International Symposium on Workload Characterization (IISWC)*. 141–151. doi:10.1109/IISWC.2012.6402918
 - [10] Chia-Hao Chang, Jihoon Han, Anand Sivasubramaniam, Vikram Sharma Mailthody, Zaid Qureshi, and Wen-Mei Hwu. 2024. GMT: GPU Orchestrated Memory Tiering for the Big Data Era. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 464–478. doi:10.1145/3620666.3651353
 - [11] Chia-Hao Chang, Adithya Kumar, and Anand Sivasubramaniam. 2021. To move or not to move? page migration for irregular applications in over-subscribed GPU memory systems with DynaMap. In *Proceedings of the 14th ACM International Conference on Systems and Storage* (Haifa, Israel) (SYSTOR '21). Association for Computing Machinery, New York, NY, USA, Article 1, 12 pages. doi:10.1145/3456727.3463766
 - [12] Shuai Che, Michael Boyer, Jiayuan Meng, David Tarjan, Jeremy W. Sheaffer, Sang-Ha Lee, and Kevin Skadron. 2009. Rodinia: A benchmark suite for heterogeneous computing. In *2009 IEEE International Symposium on Workload Characterization (IISWC)*. 44–54. doi:10.1109/IISWC.2009.5306797
 - [13] Feras Daoud, Amir Watad, and Mark Silberstein. 2016. GPUrdma: GPU-side library for high performance networking from GPU kernels. In *Proceedings of the 6th International Workshop on Runtime and Operating Systems for Supercomputers* (Kyoto, Japan) (ROSS '16). Association for Computing Machinery, New York, NY, USA, Article 6, 8 pages. doi:10.1145/2931088.2931091
 - [14] Debashis Ganguly, Ziyu Zhang, Jun Yang, and Rami Melhem. 2019. Interplay between hardware prefetcher and page eviction policy in CPU-GPU unified virtual memory. In *Proceedings of the 46th International Symposium on Computer Architecture* (Phoenix, Arizona) (ISCA '19). Association for Computing Machinery, New York, NY, USA, 224–235. doi:10.1145/3307650.3322224
 - [15] Debashis Ganguly, Ziyu Zhang, Jun Yang, and Rami Melhem. 2020. Adaptive Page Migration for Irregular Data-intensive Applications under GPU Memory Oversubscription. In *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 451–461. doi:10.1109/IPDPS47924.2020.00054
 - [16] Isaac Gelado, John E. Stone, Javier Cabezas, Sanjay Patel, Nacho Navarro, and Wen-mei W. Hwu. 2010. An asymmetric distributed shared memory model for heterogeneous parallel systems. *SIGPLAN Not.* 45, 3 (March 2010), 347–358. doi:10.1145/1735971.1736059
 - [17] NVM Express Work Group. . *NVM Express Specifications*. <https://nvmexpress.org/specifications/>
 - [18] Kshitij Gupta, Jeff A. Stuart, and John D. Owens. 2012. A study of Persistent Threads style GPU programming for GPGPU workloads. In *2012 Innovative Parallel Computing (InPar)*. 1–14. doi:10.1109/InPar.2012.6339596
 - [19] Bernhard Kerbl, Michael Kenzel, Joerg H. Mueller, Dieter Schmalstieg, and Markus Steinberger. 2018. The Broker Queue: A Fast, Linearizable FIFO Queue for Fine-Granular Work Distribution on the GPU. In *Proceedings of the 2018 International Conference on Supercomputing* (Beijing, China) (ICS '18). Association for Computing Machinery, New York, NY, USA, 76–85. doi:10.1145/3205289.3205291
 - [20] Hyojong Kim, Jaewoong Sim, Prasun Gera, Ramyad Hadidi, and Hye-soon Kim. 2020. Batch-Aware Unified Memory Management in GPUs for Irregular Workloads. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) (ASPLOS '20). Association for Computing Machinery, New York, NY, USA, 1357–1370. doi:10.1145/3373376.3378529
 - [21] Linux. 2003. *POSIX asynchronous I/O overview*. <https://man7.org/linux/man-pages/man7/aio.7.html>
 - [22] Linux. . *Linux Heterogeneous Memory Management*. <https://www.kernel.org/doc/html/latest/mm/hmm.html>
 - [23] Pak Markthub, Mehmet E. Belviranlı, Seyong Lee, Jeffrey S. Vetter, and Satoshi Matsuoka. 2018. DRAGON: Breaking GPU Memory Capacity Limits with Direct NVM Access. In *SC18: International Conference for High Performance Computing, Networking, Storage and Analysis*. 414–426. doi:10.1109/SC.2018.00035
 - [24] Jonas Markussen, Lars Bjørlykke Kristiansen, Pål Halvorsen, Halvor Kielland-Gyrud, Håkon Kvale Stensland, and Carsten Griwodz. 2021. SmartIO: Zero-Overhead Device Sharing through PCIe Networking. *ACM Transactions on Computer Systems* 38, 1–2, Article 2 (jul 2021), 78 pages. doi:10.1145/3462545
 - [25] Message Passing Interface Forum. 2025. *MPI: A Message-Passing Interface Standard Version 5.0*. <https://www.mpi-forum.org/docs/mpi-5.0/mpi50-report.pdf>
 - [26] Maged M Michael and Michael L Scott. 1996. Simple, fast, and practical non-blocking and blocking concurrent queue algorithms. In *Proceedings of the fifteenth annual ACM symposium on Principles of distributed computing*. 267–275.
 - [27] Micron. 2023. *Micron 7450 NVMe SSD*. <https://www.micron.com/products/storage/ssd/data-center-ssd/7450-ssd>
 - [28] Microsoft. 2022. *DirectX DirectStorage*. <https://devblogs.microsoft.com/directx/directstorage-api-downloads/>
 - [29] Seung Won Min, Vikram Sharma Mailthody, Zaid Qureshi, Jinjun Xiong, Eiman Ebrahimi, and Wen mei Hwu. 2021. EMOGI: Efficient Memory-access for Out-of-memory Graph-traversal In GPUs. arXiv:2006.06890 [cs.DC] <https://arxiv.org/abs/2006.06890>
 - [30] Nvidia. 2015. *GPU Pro Tip: CUDA 7 Streams Simplify Concurrency*. <https://developer.nvidia.com/blog/gpu-pro-tip-cuda-7-streams-simplify-concurrency/>
 - [31] Nvidia. 2017. *Unified Memory for CUDA Beginners*. <https://developer.nvidia.com/blog/unified-memory-cuda-beginners/>
 - [32] Nvidia. 2019. *GPUDirect Storage: A Direct Path Between Storage and GPU Memory*. <https://developer.nvidia.com/blog/gpudirect-storage/>
 - [33] Nvidia. 2020. *Controlling Data Movement to Boost Performance on the NVIDIA Ampere Architecture*. <https://developer.nvidia.com/blog/controlling-data-movement-to-boost-performance-on-ampere-architecture/>
 - [34] Nvidia. 2022. *NVIDIA Hopper Architecture In-Depth*. <https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/>
 - [35] Nvidia. 2024. *Green Contexts*. https://docs.nvidia.com/cuda/cuda-driver-api/group__CUDA__GREEN__CONTEXTS.html
 - [36] Steven G. Parker, James Bigler, Andreas Dietrich, Heiko Friedrich, Jared Hoberock, David Luebke, David McAllister, Morgan McGuire, Keith Morley, Austin Robison, and Martin Stich. 2010. OptiX: a general purpose ray tracing engine. *ACM Trans. Graph.* 29, 4, Article 66 (July 2010), 13 pages. doi:10.1145/1778765.1778803
 - [37] Zaid Qureshi, Vikram Sharma Mailthody, Isaac Gelado, Seungwon Min, Amna Masood, Jeongmin Park, Jinjun Xiong, C. J. Newburn, Dmitri Vainbrand, I-Hsin Chung, Michael Garland, William Dally, and Wen-mei Hwu. 2023. GPU-Initiated On-Demand High-Throughput Storage Access in the BaM System Architecture. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) (ASPLOS 2023). Association for Computing Machinery, New York, NY, USA, 325–339. doi:10.1145/3575693.3575748
 - [38] Samsung. 2022. *Samsung 990 Pro*. <https://semiconductor.samsung.com/consumer-storage/internal-ssd/990-pro/>
 - [39] Sagi Shahar, Shai Bergman, and Mark Silberstein. 2016. ActivePointers: a case for software address translation on GPUs. In *Proceedings of the 43rd International Symposium on Computer Architecture* (Seoul, Republic of Korea) (ISCA '16). IEEE Press, 596–608. doi:10.1109/ISCA.2016.58

- [40] Mark Silberstein, Bryan Ford, Idit Keidar, and Emmett Witchel. 2013. GPUfs: integrating a file system with GPUs. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems* (Houston, Texas, USA) (ASPLOS '13). Association for Computing Machinery, New York, NY, USA, 485–498. doi:10.1145/2451116.2451169
- [41] Markus Steinberger, Bernhard Kainz, Bernhard Kerbl, Stefan Hauswiesner, Michael Kenzel, and Dieter Schmalstieg. 2012. Softshell: dynamic scheduling on GPUs. *ACM Trans. Graph.* 31, 6, Article 161 (Nov. 2012), 11 pages. doi:10.1145/2366145.2366180
- [42] Sysbench. 2004. . <https://github.com/akopytov/sysbench>
- [43] Hung-Wei Tseng, Yang Liu, Mark Gahagan, Jing Li, Yanqin Jing, and Steven Swanson. 2015. Gullfoss: Accelerating and simplifying data movement among heterogeneous computing and storage resources. *UC San Diego: Department of Computer Science & Engineering* (2015). <https://escholarship.org/uc/item/324465dv>
- [44] Ján Veselý, Arkaprava Basu, Abhishek Bhattacharjee, Gabriel H. Loh, Mark Oskin, and Steven K. Reinhardt. 2018. Generic System Calls for GPUs. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. 843–856. doi:10.1109/ISCA.2018.00075
- [45] Jonathan D. Wapman, Sean Treichler, Serban D. Porumbescu, and John D. Owens. 2023. Harmonic CUDA: Asynchronous Programming on GPUs. In *Proceedings of the 14th International Workshop on Programming Models and Applications for Multicores and Manycores* (Montreal, QC, Canada) (PMAM'23). Association for Computing Machinery, New York, NY, USA, 39–49. doi:10.1145/3582514.3582517
- [46] Shucan Xiao and Wu-chun Feng. 2010. Inter-block GPU communication via fast barrier synchronization. In *2010 IEEE International Symposium on Parallel and Distributed Processing (IPDPS)*. 1–12. doi:10.1109/IPDPS.2010.5470477
- [47] Zi Yan, Daniel Lustig, David Nellans, and Abhishek Bhattacharjee. 2019. Nimble Page Management for Tiered Memory Systems. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) (ASPLOS '19). Association for Computing Machinery, New York, NY, USA, 331–345. doi:10.1145/3297858.3304024
- [48] Zhuoping Yang, Jinming Zhuang, Xingzhen Chen, Alex K. Jones, and Peipei Zhou. 2025. AGILE: Lightweight and Efficient Asynchronous GPU-SSD Integration. arXiv:2504.19365 [cs.DC] <https://arxiv.org/abs/2504.19365>
- [49] Haoyang Zhang, Yirui Zhou, Yuqi Xue, Yiqi Liu, and Jian Huang. 2023. G10: Enabling An Efficient Unified GPU Memory and Storage Architecture with Smart Tensor Migrations. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture* (Toronto, ON, Canada) (MICRO '23). Association for Computing Machinery, New York, NY, USA, 395–410. doi:10.1145/3613424.3614309
- [50] Jie Zhang, David Donofrio, John Shalf, Mahmut T. Kandemir, and Myoungsoo Jung. 2015. NVMMU: A Non-volatile Memory Management Unit for Heterogeneous GPU-SSD Architectures. In *2015 International Conference on Parallel Architecture and Compilation (PACT)*. 13–24. doi:10.1109/PACT.2015.43
- [51] Jie Zhang, Miryeong Kwon, Hyojong Kim, Hyesoon Kim, and Myoungsoo Jung. 2019. FlashGPU: Placing New Flash Next to GPU Cores. In *Proceedings of the 56th Annual Design Automation Conference 2019* (Las Vegas, NV, USA) (DAC '19). Association for Computing Machinery, New York, NY, USA, Article 156, 6 pages. doi:10.1145/3316781.3317827
- [52] Lingqi Zhang, Mohamed Wahib, Peng Chen, Jintao Meng, Xiao Wang, Toshio Endo, and Satoshi Matsuoka. 2023. PERKS: a Locality-Optimized Execution Model for Iterative Memory-bound GPU Applications. In *Proceedings of the 37th ACM International Conference on Supercomputing* (Orlando, FL, USA) (ICS '23). Association for Computing Machinery, New York, NY, USA, 167–179. doi:10.1145/3577193.3593705